



Digital logic design course
Final project

Muplitlier! Project report

Prof. [Hatam Abdoli](#)

Amir Mohammad Fazeli 40212358030

Pouya Tavakoli 40212358011



TABLE OF CONTENTS

[Introduction](#)

[Modules Overview](#)

[Full Adder](#)

[Adder 8bit](#)

[mul 1x1](#)

[Mul 2x2](#)

[Mul 4x4 with 1x1](#)

[Mul 4x4 with 2x2](#)

[Shift left x](#)

[3-2 compressor](#)

[4-2 compressor](#)

[Mul 2x2_opt](#)

[Cost Optimization](#)

[Gate Reduction](#)

[Parallel Computing](#)

[mul 4x4 with 1x1_opt](#)

[Test Benches and Waveforms](#)

[Full Adder](#)

[Adder 8bit](#)

[Mul 1x1](#)

[Mul 2x2](#)

[Mul 4x4 with 1x1](#)

[Mul 4x4 with 2x2](#)

[Shift left x](#)

[3-2 compressor](#)

[4-2 compressor](#)

[Mul 2x2_opt](#)

[mul 4x4 with 2x2_opt](#)



Approximate computing

Truncation

[How It Works](#)

[Advantages](#)

[Disadvantages](#)

[Use Cases](#)

Rounding

[How It Works](#)

[Advantages](#)

[Disadvantages](#)

[Use Cases](#)

Logarithmic Approximation

[How It Works](#)

[Advantages](#)

[Disadvantages](#)

[Use Cases](#)

Approximate Partial Product Generation

[How It Works](#)

[Advantages](#)

[Disadvantages](#)

[Use Cases](#)

Approximate Compressors

[How It Works](#)

[Advantages](#)

[Disadvantages](#)

[Use Cases](#)



What Have We Used?

Why Compressors?

Key Reasons for Choosing Compressors

Efficiency in Partial Product Reduction

Cost and Power Optimization

Flexibility and Scalability

Extensive Research and Proven Effectiveness

Optimal Trade-off Between Accuracy and Performance

Why This Approach?

Wide Applicability

Proven Results

Ease of Implementation

Alignment with Project Goals

Statistics

Total Cases (256)

Correct Cases (200)

Mean Absolute Error (MAE = 4.375)

Mean Squared Error (MSE = 156.75)

Max Absolute Error (70)

Mean Relative Error ($\approx 4.42\%$)

Error Distribution

Conclusion

References



Introduction

This report details the development of an approximate circuit design using VHDL (VHSIC Hardware Description Language), a powerful language used for describing the behavior and structure of electronic systems.

In digital systems, the performance of a multiplier is critical, as it is often the slowest element in the overall architecture. Multipliers typically consume significant area and power, and they are associated with long latency, which can adversely affect the performance of the entire system. As such, optimizing multiplier design is essential for enhancing the efficiency and speed of digital circuits.

An $n \times n$ multiplication operation is conventionally divided into three operational phases: Partial Product Generation, Carry-Free Reduction of Partial Products, and Carry Propagating Addition. Among these phases, the Partial Product Reduction phase has garnered the most attention in research and design efforts focused on parallel multipliers. This is primarily due to its substantial contribution to area and power consumption, making it a critical target for optimization.

This report will explore the design and implementation of an approximate multiplier circuit using VHDL, emphasizing the strategies employed to reduce area and power consumption while maintaining acceptable performance levels. By leveraging approximate computing techniques, this project aims to demonstrate that it is possible to achieve significant improvements in efficiency without compromising the overall functionality of the system.



Modules Overview

The project consists of several key modules that work together to implement the exact and approximate multipliers design.

Full_Adder

The full adder module is a fundamental building block for arithmetic operations. It takes three input bits (two significant bits and a carry-in) and produces a sum and a carry-out. It is used in making n bit adders as a base component.

Adder_8bit

The 8 bit adder is made of multiple full adders and as the name states is used to sum two 8 bit numbers

mul_1x1

As simple as the name suggests this module is a one by one multiplier and its job is to multiply two bits together using a simple AND gate.

Mul_2x2

This module gets two 2it numbers as input and multiplies them together.

Mul_4x4_with_1x1

This module implements a 4x4 multiplication operation using the 1x1 multiplier as a fundamental component. In this hierarchical structure, the 1x1 multipliers are employed to generate the partial products necessary for the multiplication process.



Mul_4x4_with_2x2

Similar to the previous module this module also has a hierarchical structure and multiplies two 4 bit numbers using 2 1x1 multipliers as the fundamental component for generating partial products.

Shift_left_x

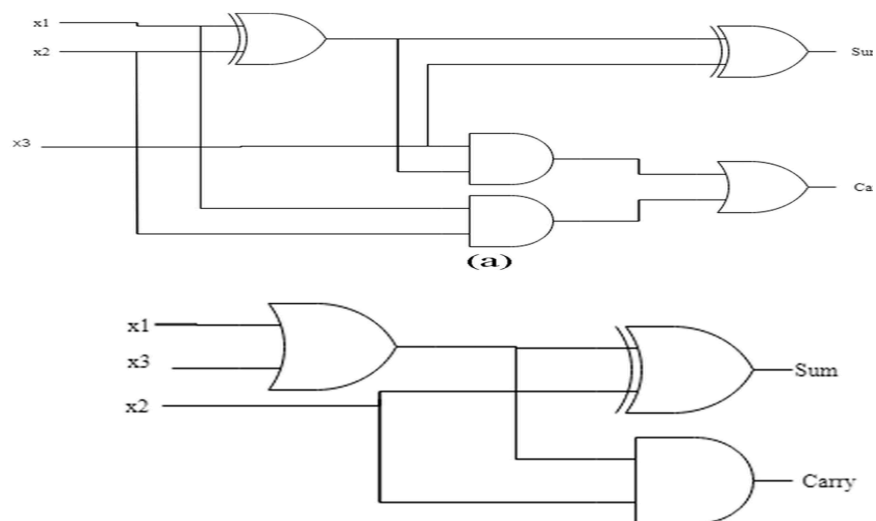
These modules are used to shift partial products by x bits, the shift_left_0 module may seem weird because it may look like it is not even doing anything but its main duty is to make zero padding for the 4 bit number to become 8 bit. All three shift register modules used in this project get 4 bit numbers as input and output proper 8 bits.

3-2 compressor

A 3-2 compressor is the fundamental building block of a multiplier. It is used to efficiently reduce the partial products during multiplication to compress multiple bit inputs into fewer outputs.

It is also known as **full adder** because it performs similarly as full adder, but in partial product reduction, it is often referred to as a compressor.

Below is a head to head comparison between a normal full adder and approximated compressor.

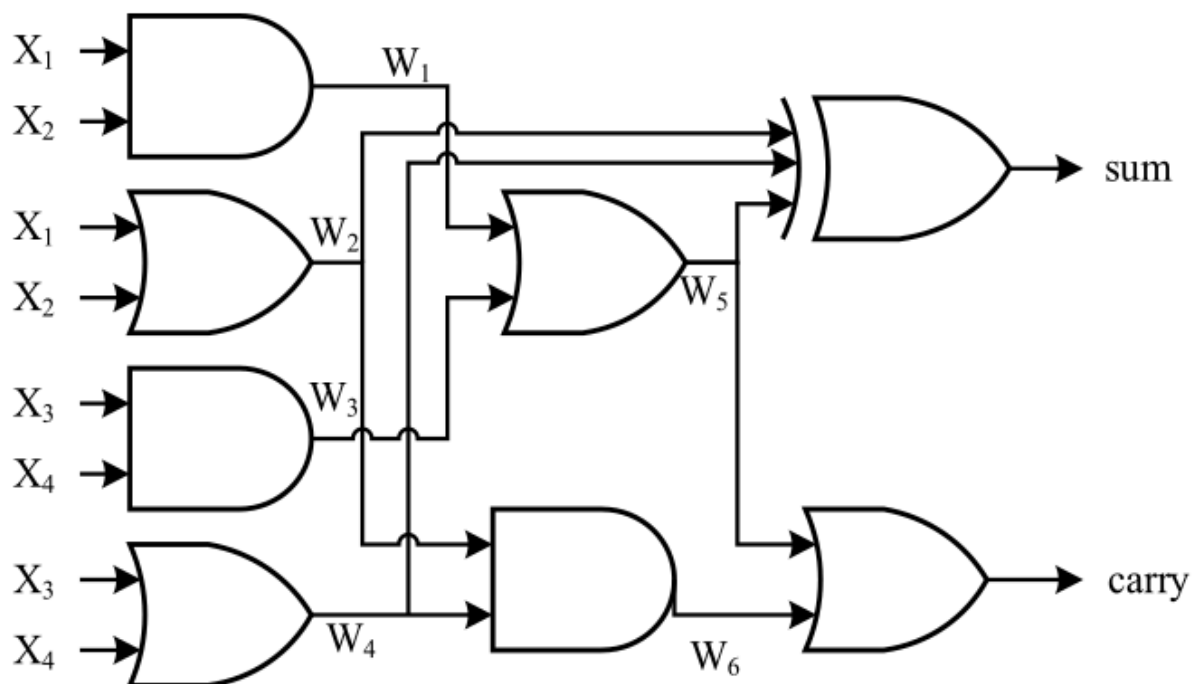


4-2 compressor

In designing high performance multipliers 4-2 compressor plays a significant role just like a 3-2 compressor, It is used to reduce partial products during multiplication process.

The 4-2 compressor has 4 inputs and 2 outputs. In comparison with a standard full adder which is connected in series, 4-2 compressor not only costs lower but also a reduced propagation delay.

Below is a full implementation of 4-2 approximate compressor.





Mul_2x2_opt

This 2x2 multiplier module is designed to significantly reduce both cost and propagation delay while maintaining high accuracy. By optimizing the logic structure, we have achieved a more efficient and compact design compared to a conventional 2x2 multiplier.

Cost Optimization

The optimized 2x2 multiplier reduces the number of logic gates required, resulting in a more cost-effective solution. Here's how it compares to a standard 2x2 multiplier:

Gate Reduction

The design eliminates 3 gates compared to a traditional 2x2 multiplier, reducing the overall hardware complexity.

This reduction in gates directly translates to lower area usage and manufacturing costs.

Parallel Computing

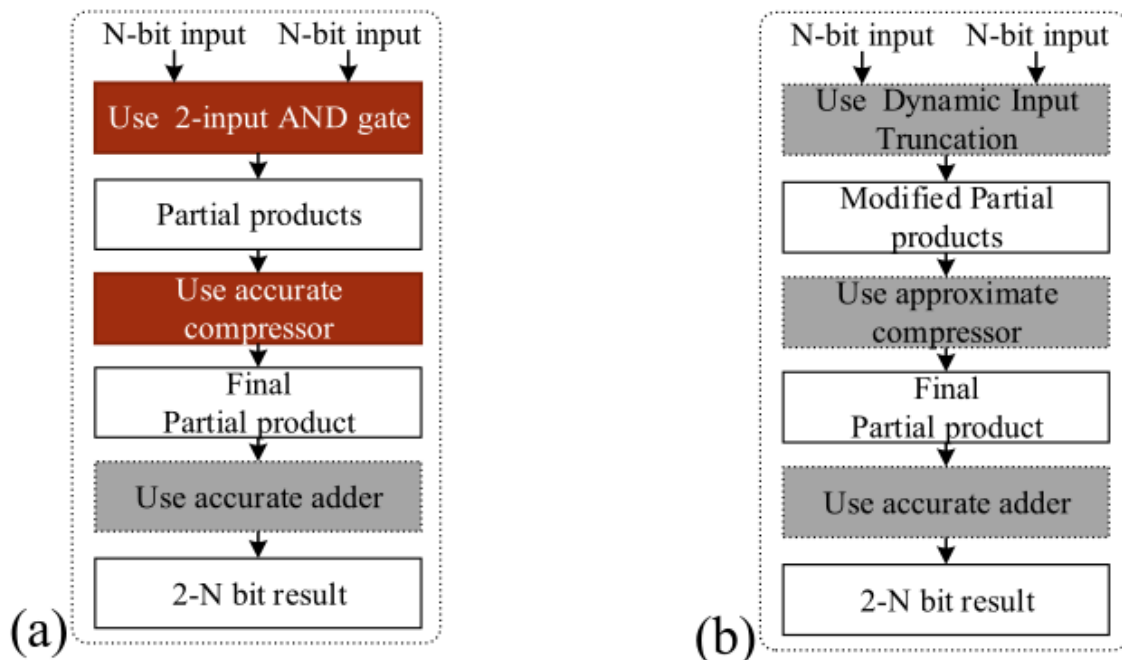
By leveraging parallel computation, the propagation delay is minimized, improving the overall speed of the multiplier.

This approach ensures that the multiplier operates faster while consuming fewer resources.

mul_4x4_with_1x1_opt

How does it work?

Below image gives a clear understanding of how traditional(a) and approximate(b) multiplication works.



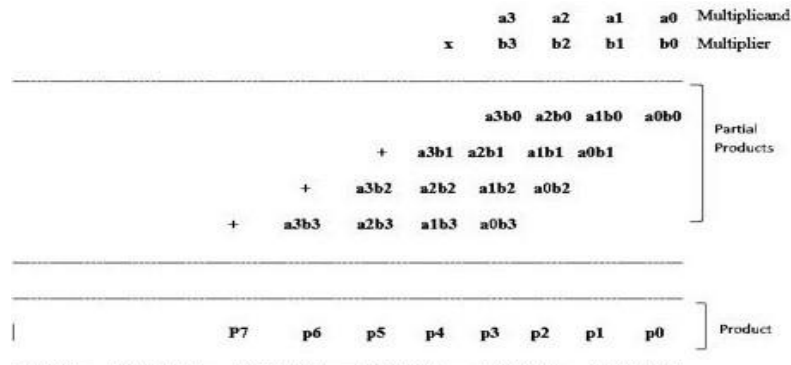
We often try to approximate the result by using compressors which were explained earlier.

In a 4x4 multiplication the result would be an 8 bit product.

Dynamic input truncation wasn't used in this project due to more simplicity.

First we need to calculate all partial products.

Second, we use different methods to reduce partial products.



Above is a schematic of how a multiplication is done.

To reduce the partial products we use 3-2 and 4-2 compressors and some AND & XOR gates.

a0b0 is the first product.

a1b0 & a0b1 will have a sum and carry which will be calculated by XOR & AND gate.

a2b0 & a1b1 & a0b2 will be calculated by a 3-2 compressor.

a3b0 & a2b1 & a1b2 & a0b3 will be calculated by a 4-2 compressor.

a3b1 & a2b2 & a1b3 will be calculated by a 3-2 compressor.

a3b2 & a2b3 will have a sum and carry which will be calculated by XOR & AND gates.

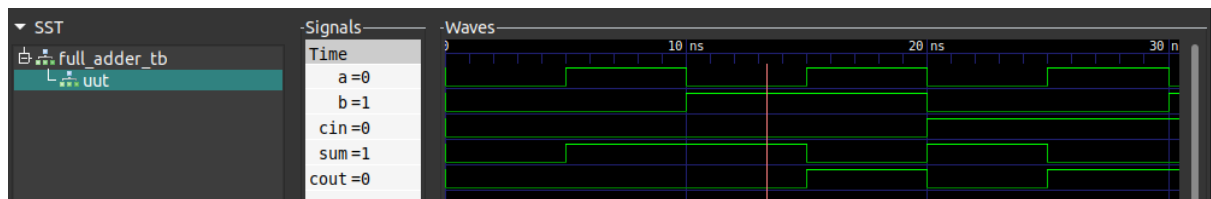
a3b3 is the last product.

Then we use an 8 bit adder to sum all the reduced partial products and their carries.

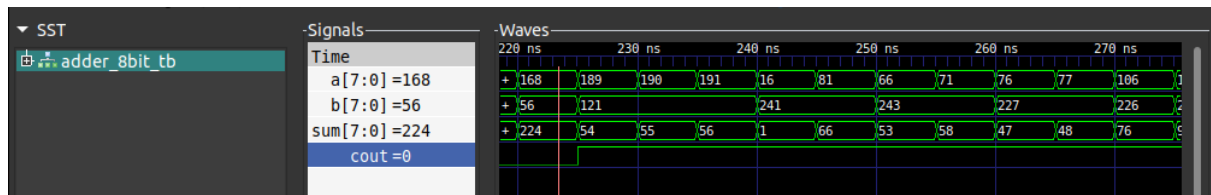
Test Benches and Waveforms

Each module in this project is accompanied by a dedicated test bench to ensure that all components function correctly and meet the design specifications. The test benches are designed to apply a variety of input scenarios to each module, allowing for comprehensive verification of their behavior and performance. In addition to the test benches, each module also includes its own simulation script. The simulation script automates the process of managing the workspace, compiling VHDL files, and running simulations for each module. It ensures that all components are syntax-checked and elaborated correctly, generating waveform data in a VCD file for visual analysis. Additionally, the script provides an option to open GTKWave for immediate waveform visualization, facilitating efficient debugging and verification of module functionality.

Full_Addder

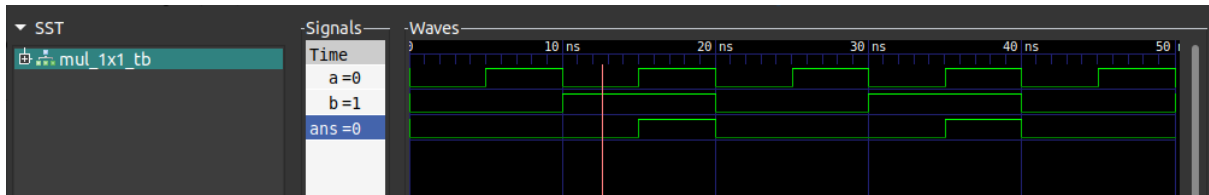


Adder_8bit

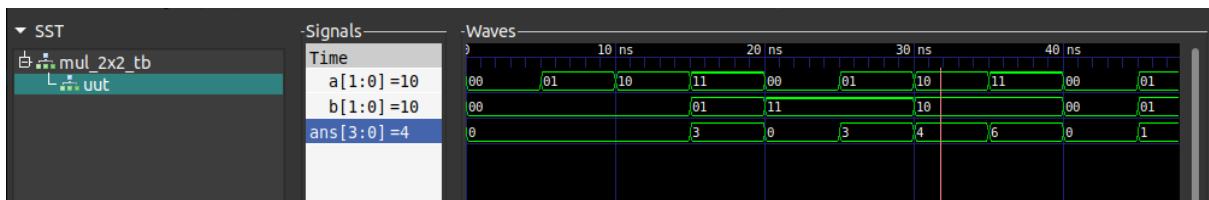




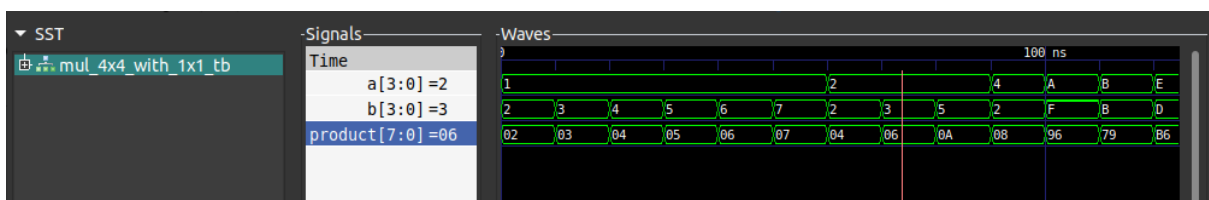
Mul_1x1



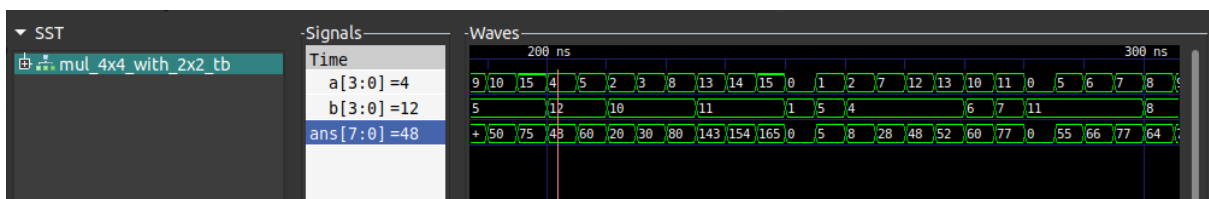
Mul_2x2



Mul_4x4_with_1x1

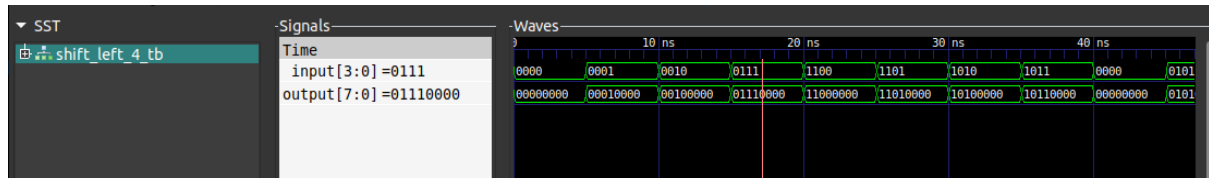


Mul_4x4_with_2x2

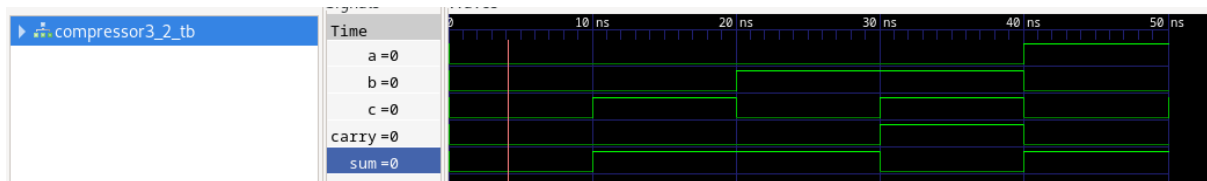




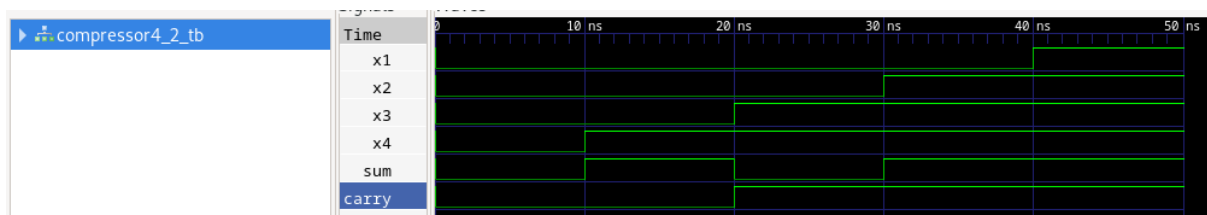
Shift_left_x



3-2 compressor

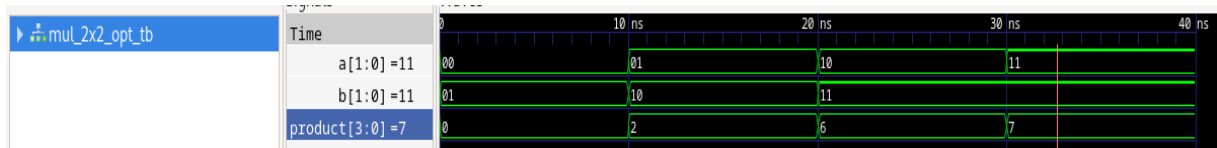


4-2 compressor

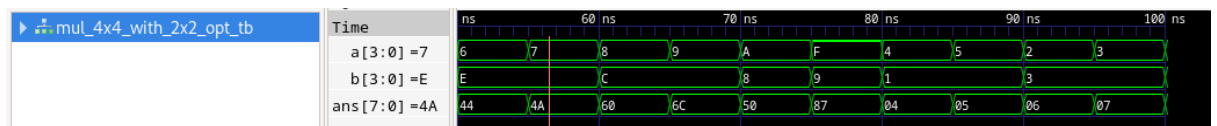




Mul_2x2_opt



mul_4x4_with 2x2_opt





Approximate computing

Approximate computing is a design paradigm that trades off computational accuracy for improvements in performance, power efficiency, and area savings. This approach is particularly useful in applications where exact precision is not critical, such as image processing, machine learning, and multimedia systems. In multiplication, several approximation techniques have been developed to reduce complexity and energy consumption while maintaining acceptable accuracy. Below is an explanation of the most common approximation techniques used in multipliers.

Truncation

Truncation is one of the simplest approximation techniques. It involves discarding the least significant bits (LSBs) of the partial products or the final result.

How It Works

During multiplication, only the most significant bits (MSBs) are retained, and the LSBs are truncated.

For example, in a 16-bit multiplier, only the upper 8 bits might be retained, and the lower 8 bits are discarded.

Advantages

Reduces hardware complexity and power consumption.

Simplifies the design by eliminating the need for LSB computation.

Disadvantages

Introduces significant error, especially for small input values.

Use Cases

Image and video processing, where small errors are often imperceptible.



Rounding

Rounding is a more controlled form of approximation compared to truncation. It involves rounding the result to the nearest representable value.

How It Works

After computing the partial products, the result is rounded to a fixed number of bits.

For example, a 16-bit result might be rounded to 8 bits by adding a rounding constant and then truncating the LSBs.

Advantages

Provides better accuracy than truncation.

Reduces hardware complexity compared to exact computation.

Disadvantages

Slightly more complex than truncation due to the need for rounding logic.

Still introduces some error, though less than truncation.

Use Cases

Digital signal processing (DSP) and audio processing, where moderate precision is acceptable.



Logarithmic Approximation

Logarithmic approximation leverages the mathematical properties of logarithms to simplify multiplication.

How It Works

Multiplication is transformed into addition using the logarithmic identity:

$$\log(A \times B) = \log(A) + \log(B)$$

The result is then converted back using an anti logarithmic function.

Advantages

Reduces multiplication to addition, which is simpler and faster.

Significant power and area savings.

Disadvantages

Requires precomputed logarithmic and antilogarithmic tables.

Introduces approximation errors due to the logarithmic transformation.

Use Cases

Machine learning and neural networks, where exact precision is not critical.



Approximate Partial Product Generation

This technique involves approximating the generation of partial products in a multiplier.

How It Works

Instead of computing all partial products exactly, some are approximated or ignored.

For example, in a 4x4 multiplier, only the most significant partial products might be computed, and the rest are set to zero.

Advantages

Reduces the number of partial products, simplifying the design.

Improves speed and reduces power consumption.

Disadvantages

Introduces errors, especially for large input values.

Requires careful design to balance accuracy and performance.

Use Cases

Image processing and multimedia applications, where small errors are tolerable.



Approximate Compressors

Approximate compressors are used in partial product reduction to simplify the addition process.

How It Works

Traditional compressors (e.g., 3:2 or 4:2 compressors) are replaced with approximate versions that use fewer gates or simpler logic.

For example, an approximate 4:2 compressor might compute the carry-out incorrectly in some cases to save area and power.

Advantages

- Reduces the complexity of partial product reduction.

- Improves speed and energy efficiency.

Disadvantages

- Introduces errors in the partial product reduction process.

- Requires careful error analysis to ensure acceptable accuracy.

Use Cases

- High-performance multipliers in DSP and machine learning.



What Have We Used?

In this project, we have employed partial product reduction using compressors as the core technique for designing an approximate multiplier. This approach focuses on simplifying the multiplication process by reducing the number of partial products efficiently while maintaining an acceptable level of accuracy.

Why Compressors?

In the realm of approximate multipliers, compressors have emerged as one of the most researched and widely adopted components. Their popularity stems from their ability to significantly reduce hardware complexity, power consumption, and critical path delay, making them an ideal choice for energy-efficient and high-performance designs.

Key Reasons for Choosing Compressors

Efficiency in Partial Product Reduction

Compressors excel at reducing the number of partial products in a multiplication operation. By compressing multiple input bits into fewer outputs, they simplify the addition process, which is a critical step in multipliers.

Cost and Power Optimization

Compressors are inherently designed to minimize the number of logic gates and interconnects, leading to reduced area and power consumption. This makes them highly suitable for low-power and resource-constrained applications.

Flexibility and Scalability

Compressors can be easily adapted to different levels of approximation, allowing designers to strike a balance between accuracy and efficiency. They can also be cascaded to handle larger bit-widths, making them scalable for various applications.



Extensive Research and Proven Effectiveness

Compressors have been extensively studied in the context of approximate computing, with numerous research articles demonstrating their effectiveness in improving the performance of multipliers. This wealth of knowledge provides a strong foundation for implementing and optimizing compressor-based designs.

Optimal Trade-off Between Accuracy and Performance

By carefully designing the compressor logic, it is possible to achieve a multiplier that meets the accuracy requirements of the target application while significantly improving speed and energy efficiency.

Why This Approach?

After a thorough review of existing literature and techniques in the field of approximate multipliers, we concluded that compressors offer the best balance between efficiency, simplicity, and effectiveness. Here's why:

Wide Applicability

Compressors are versatile and can be applied to a variety of applications, from image and signal processing to machine learning and neural networks, where exact precision is often unnecessary.

Proven Results

Numerous studies have demonstrated that compressor-based multipliers achieve significant improvements in power consumption, area, and delay compared to traditional designs, without compromising the overall functionality of the system.



Ease of Implementation

Compressors are relatively straightforward to implement and integrate into existing multiplier architectures, making them a practical choice for real-world applications.

Alignment with Project Goals

Our primary objective was to design a multiplier that is both energy-efficient and high-performing. Compressors align perfectly with this goal, as they enable us to achieve substantial reductions in power and area while maintaining acceptable accuracy.



Statistics

The performance of the approximate 4×4 multiplier was quantitatively assessed by comparing its outputs with those of an exact multiplier over all possible input combinations. In our test, 256 cases were evaluated, and the comparison yielded the following key statistics

Total Cases (256)

Every possible input combination for a 4×4 multiplier was exercised. This comprehensive testing ensures that the evaluation covers the entire operating range of the multiplier.

Correct Cases (200)

Out of 256 comparisons, 200 cases produced the exact correct output, corresponding to approximately 78% accuracy. This high rate of correct outputs demonstrates that the approximate multiplier matches the exact results in the majority of instances.

Mean Absolute Error (MAE = 4.375)

The MAE indicates that, on average, the magnitude of the error between the approximate and exact outputs is 4.375 units. Because MAE is scale-dependent, it provides an intuitive measure of how far the approximate results deviate from the exact ones in absolute terms.

Mean Squared Error (MSE = 156.75)

MSE is computed by averaging the squares of the individual errors. The larger value here suggests that while most cases exhibit low errors, a few instances have significantly higher discrepancies. The squaring process in MSE emphasizes these larger deviations, making it a useful metric to identify potential worst-case issues.

Max Absolute Error (70)

The worst-case error observed was 70 units. Although this extreme value occurs only in a minority of cases, it is critical for applications where even occasional large errors could be problematic.

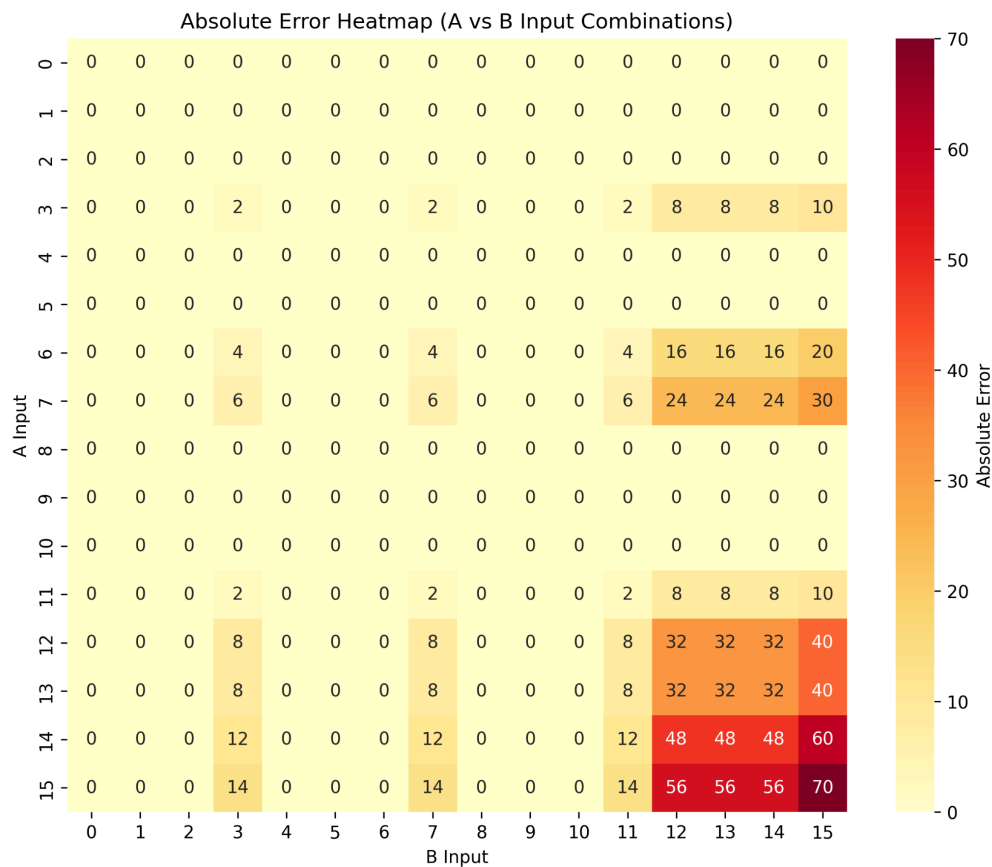


Mean Relative Error ($\approx 4.42\%$)

By normalizing the absolute errors relative to the exact outputs, the mean relative error of about 4.42% provides a scale-independent perspective. This percentage allows for a more intuitive understanding of the error magnitude relative to the true values, facilitating comparisons with similar systems.

Error Distribution

An analysis of the error distribution (typically visualized using a histogram) reveals that while most errors are clustered near zero, there are some outliers contributing to the higher MSE and the observed maximum error. This pattern suggests that the approximate multiplier performs very well for a large fraction of the input space but may exhibit notable deviations for specific input combinations.





Conclusion

In this project, we successfully designed and implemented an approximate multiplier using VHDL, focusing on optimizing area, power consumption, and computational efficiency. Through the use of compressors, we effectively reduced the number of partial products while maintaining an acceptable level of accuracy. The approach demonstrated that approximate computing techniques can significantly enhance the performance of digital circuits without severely compromising the quality of results.

Our design leveraged multiple key components, including full adders, shift registers, and optimized multipliers, to achieve a structured and efficient implementation. By comparing traditional and approximate multipliers, we highlighted the advantages of reducing logic complexity, minimizing propagation delay, and improving overall power efficiency. Test benches and simulations validated the correctness of our design.

The statistical evaluation demonstrates that the VHDL-based approximate 4×4 multiplier achieves a high rate of exact matches (78% correct cases) and maintains a low average error (MAE of 4.375) across all tests. However, the presence of a few significant errors (with a maximum error of 70) and a relatively high MSE indicate that, while the average performance is strong, attention must be given to those worst-case scenarios.

Ultimately, this project underscores the potential of approximate computing in hardware design, showcasing how strategic optimizations can lead to more resource-efficient digital systems.



References

- [1] IEEE : <https://www.ieee.org/>. <https://ieeexplore.ieee.org/>
- [2] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with VHDL Design, 4th ed.
- [3] N. V. Toan and J.-G. Lee, "FPGA-based Multi-Level Approximate Multipliers for High-Performance Error-Resilient Applications," IEEE Access, doi: 10.1109/ACCESS.2017.DOI.
- [4] P. Darshini, "Design and Analysis of Approximate Multipliers," doi: 10.13140/RG.2.2.10010.52160.
- [5] HARDI Electronics AB, VHDL Handbook
- [6] L. Qian, C. Wang, W. Liu, F. Lombardi, and J. Han, "Design and Evaluation of An Approximate Wallace-Booth Multiplier," [conference paper], College of EIE, Nanjing University of Aeronautics and Astronautics; Department of ECE, Northeastern University; Department of ECE, University of Alberta.
- [7] H. Jiang, C. Liu, L. Liu, F. Lombardi, and J. Han, "A Review, Classification, and Comparative Evaluation of Approximate Arithmetic Circuits."